1. IDENTIFICACIÓN

Nombre: Árbol Binario de Búsqueda (BST)

Definición: Estructura de datos arbórea donde cada nodo tiene **máximo dos hijos** y cumple la **propiedad de orden**: los valores menores van al subárbol izquierdo y los mayores al derecho. Permite mantener datos ordenados dinámicamente.

2. ANÁLISIS DE COMPLEJIDAD (BIG O)

Operación	Peor caso	Caso promedio
Búsqueda	O(n)	O(log n)
Inserción	O(n)	O(log n)
Eliminación	O(n)	O(log n)

En el **peor caso**, el árbol puede degenerarse como una lista lineal, requiriendo tiempo proporcional al número de nodos (n). En el **caso promedio**, si el árbol está balanceado, la altura es logarítmica (log n), resultando en operaciones mucho más eficientes.

3. CARACTERÍSTICAS MECÁNICAS

- Cada nodo contiene un **valor**, una referencia al hijo **izquierdo** y una referencia al hijo **derecho**.
- La **inserción** se basa en comparaciones sucesivas para determinar la posición correcta del nuevo nodo.
- La **eliminación** considera tres casos principales:
  - Nodo sin hijos.
  - Nodo con un solo hijo.
  - Nodo con dos hijos (se reemplaza por su sucesor in-order).
- El recorrido **InOrder** (izquierdo, raíz, derecho) devuelve los elementos del árbol en **orden ascendente**.

4. VENTAJAS Y LIMITACIONES

Ventaja Principal:

Realiza búsquedas, inserciones y eliminaciones de manera muy **eficiente** (promedio O(log n)) cuando el árbol está **balanceado**.

Limitación Principal:

El rendimiento puede degradarse significativamente, acercándose a O(n) en el peor caso, si el árbol se **desbalancea** debido a un orden de inserción/eliminación específico.

5. CASO DE USO EN LA INDUSTRIA

Los Árboles Binarios de Búsqueda, o estructuras basadas en principios similares, son fundamentales en:

- **Índices en bases de datos:** Para acelerar las consultas y la recuperación de datos.
- **Sistemas de búsqueda:** Implementación de búsquedas eficientes en grandes volúmenes de información.
- **Motores SQL:** Para optimizar la ejecución de consultas complejas.
- **Algoritmos de ordenamiento y búsqueda eficientes.**

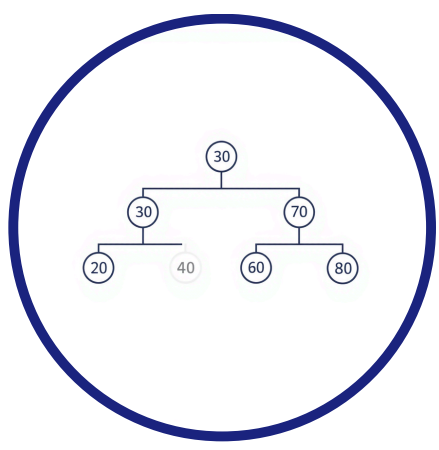
6. DIAGRAMA DEL ÁRBOL

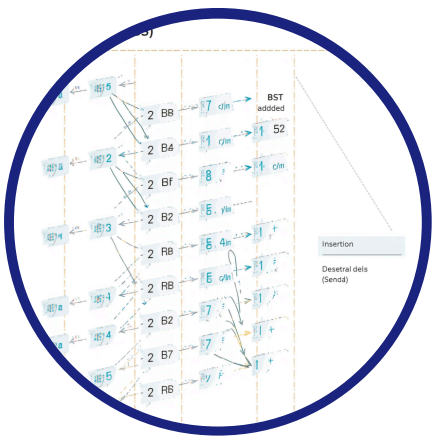
ÁRBOL BINARIO

Ejemplo de un Árbol Binario de Búsqueda (BST)

ESTRUCTURA

Un BST organiza nodos de forma jerárquica, facilitando la búsqueda y la inserción de datos.



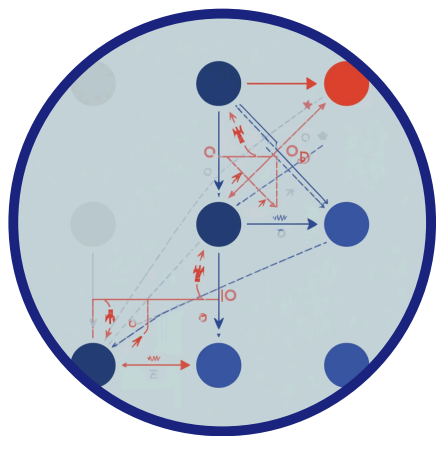


BÚSQUEDA

Facilita la búsqueda rápida de valores específicos en comparación con estructuras lineales.

INSERCIÓN

Nuevos nodos se añaden fácilmente, manteniendo el orden del árbol para eficiencias futuras.



Ficha Técnica – Árbol AVL

Identificación

Árbol binario de búsqueda auto-balanceado creado por Adelson-Velsky y Landis. Garantiza que la diferencia de altura entre subárbol izquierdo y derecho sea máximo ±1.

Fundamento Teórico

Cada nodo almacena: valor, referencia izquierda, referencia derecha y altura. El factor de balance se calcula como altura izquierda menos altura derecha.

Si el valor absoluto es mayor a 1 se aplican rotaciones: izquierda, derecha, doble izquierda-derecha y doble derecha-izquierda.

Esto mantiene la altura aproximadamente log(n).

Complejidad

Operación	Complejidad
Búsqueda	O(log n)
Inserción	O(log n)
Eliminación	O(log n)

Ventajas

Altura garantizada, rendimiento estable, ideal para sistemas críticos.

Limitaciones

Implementación compleja y mayor costo en rotaciones.

Uso en Industria

Bases de datos, sistemas financieros, aplicaciones en tiempo real.

Ficha Técnica – Árbol B / B+

Identificación

Estructura balanceada multi-camino diseñada para almacenamiento en disco.

Diferencia

Árbol B: datos en nodos internos y hojas. Árbol B+: datos solo en hojas y hojas enlazadas.

Fundamento Teórico

Permite múltiples claves por nodo, minimiza accesos a disco y mantiene altura baja.

Complejidad

Operación	Complejidad
Búsqueda	O(log n)
Inserción	O(log n)
Eliminación	O(log n)

Ventajas

Óptimo para grandes volúmenes y bases de datos.

Limitaciones

Complejo e innecesario para estructuras pequeñas.

Ficha Técnica – Trie

Identificación

Estructura para almacenamiento eficiente de cadenas.

Fundamento Teórico

Cada nodo representa un carácter y las palabras comparten prefijos.

Complejidad

Operación	Complejidad
Búsqueda	O(m)
Inserción	O(m)
Eliminación	O(m)

Ventajas

Excelente para autocompletado.

Limitaciones

Alto consumo de memoria.

Ficha Técnica – Heap

Identificación

Árbol binario completo con orden parcial.

Tipos

Min-Heap y Max-Heap.

Complejidad

Operación	Complejidad
Insertar	O(log n)
Eliminar raíz	O(log n)
Obtener min/max	O(1)

Ventajas

Base de Priority Queue, selección rápida.

Limitaciones

No eficiente para búsquedas arbitrarias.